

# Predictive Maintenance for Continuous Production

From reactive downtime to proactive line management. A working reference implementation with all numbers measured from real telemetry.

Annual cost of unplanned downtime

# \$33M

1,500 hours per year across eight plants, at \$22,000 per hour, plus expedited freight and SLA penalties.

PER HOUR LOST

## \$22K

Entire line output

HOURS PER YEAR

## 1,500

All eight plants

PLUS FREIGHT & SLA

## \$10M+

Expedited parts, penalties

# Why Current Monitoring **Misses the Window**

Plant managers see downtime after it happens. Telemetry exists but is not watched in time.

Traditional RMS vibration alarms miss the critical early phase where intervention is possible.

## The Physics

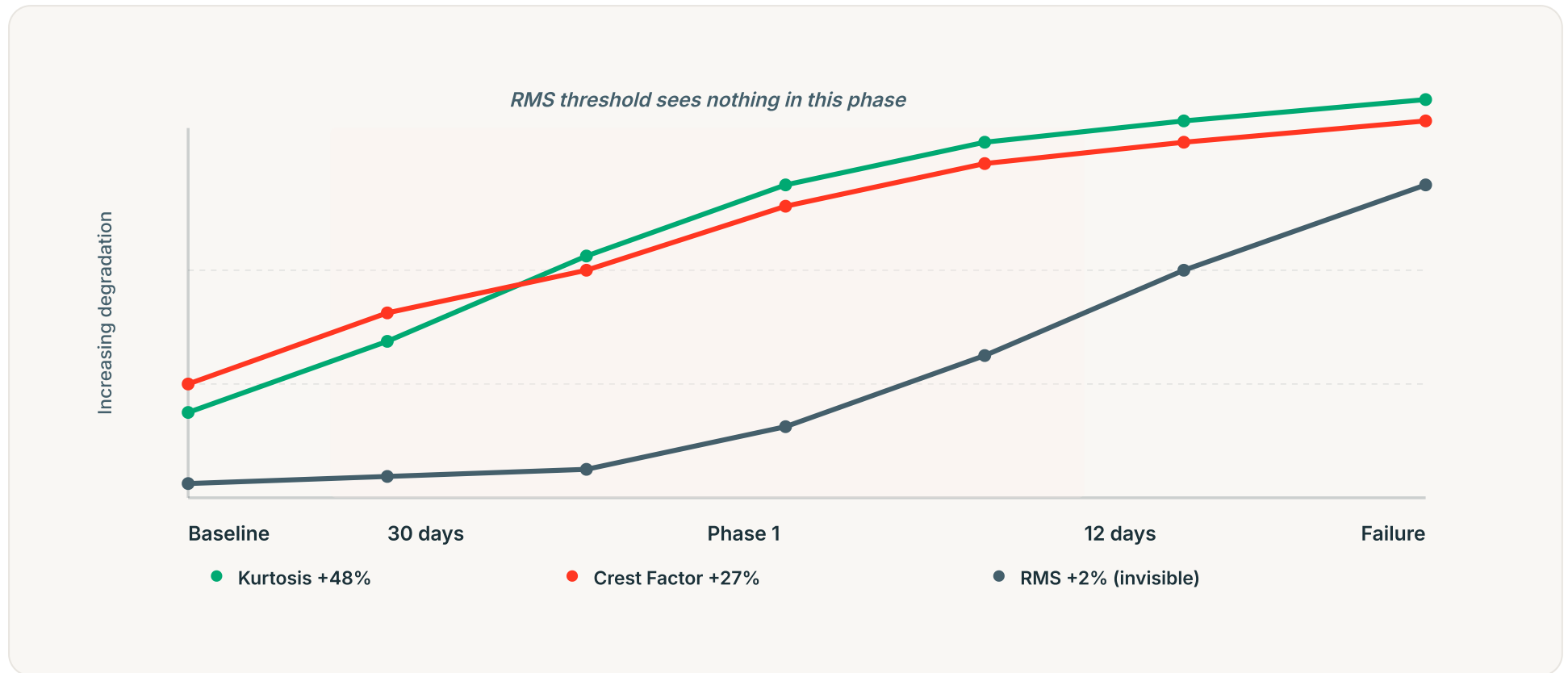
A bearing defect is impulsive. Waveform statistics (crest factor, kurtosis) rise sharply in early degradation while overall energy barely moves.

## The Result

An RMS threshold tuned to minimize false positives is blind to the first 18 days. By the time it triggers, options are gone.

# What We Measure: The Early Window

Three waveform statistics over 30 days before bearing failure. Phase 1 (30-12 days out) is where the model detects and RMS is blind.



Reference build: bearing degradation over 30 days. Kurtosis and crest factor rise 27-48% in Phase 1 while RMS stays nearly flat. Phase 2 (final 12 days): all three metrics climb sharply.

# Reference Implementation on Ironbark Resources

We built and validated the complete system end to end. The numbers below are real measured output from that build. The architecture transfers directly to discrete manufacturing.

- ▶ 151 instrument tags across 25 assets (crushers, screens, conveyors, cyclones, stacker, bins)
- ▶ 21.7 million sensor readings over a single 24-hour window
- ▶ 120 days of historical condition-monitoring data for model training
- ▶ Live mass balance reconciling to 0.01% error on a 2,500-tonne-per-hour feed

This is a continuous-process plant (mining), not a discrete manufacturing line. The architecture transfers directly: same lakehouse structure, same serverless serving layer, same model training loop, same audit trail.

# Early Detection: 95.8% vs 14.8%

In the 30-to-12-day window before bearing failure, recall on degrading readings:



Our Model

**95.8%**

Catches degrading readings in the actionable window.  
F1 0.988, validated leave-one-asset-out.



RMS Threshold

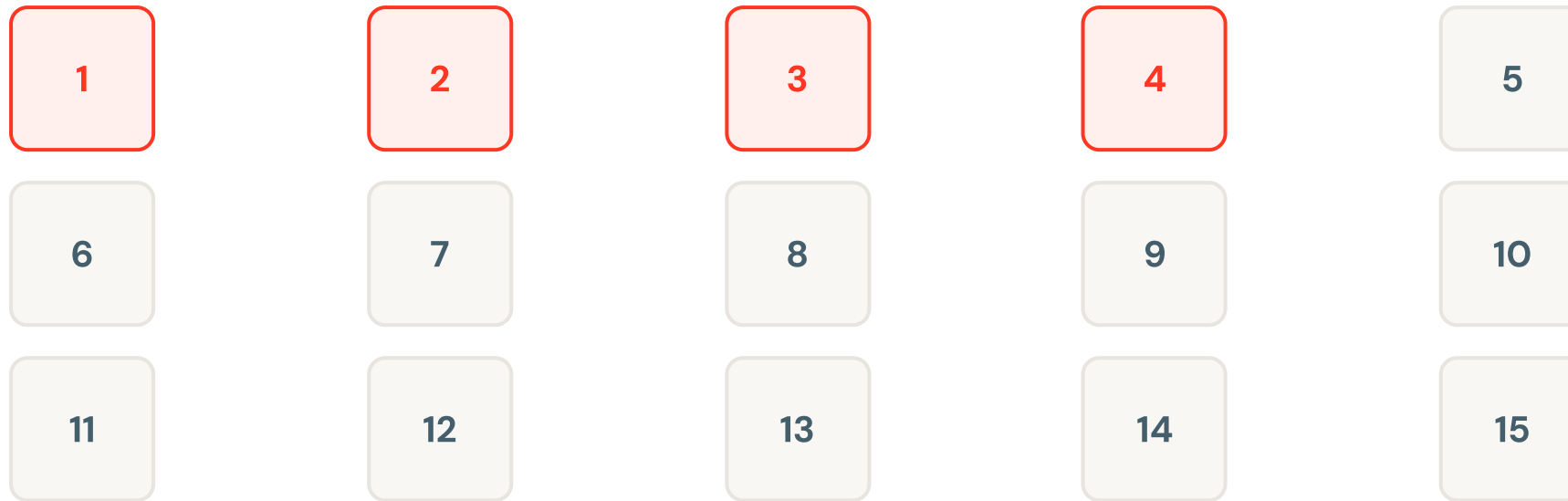
**14.8%**

Tuned to minimize false positives. For 3 of 4 failures, it  
saw nothing until the final 12 days.

The difference is roughly 18 extra days to move a failure from unplanned emergency to scheduled maintenance, with the production plan intact.

# Precision: Zero False Positives

Batch scoring across all 15 vibration sensors over the 30-day window:



☒ Flagged (model detected): 4 sensors

☐ Clear (no signal): 11 sensors

Model score

**above 0.7 on exactly the 4 that failed, 0.000 on the other 11**

A control room that cries wolf gets ignored. The model does not cry wolf. It caught what failed and saw nothing on what did not.

LIVE ENVIRONMENT

# Demo time

Everything from here is running live on your data model: streaming ingest, the trained model scoring in real time, and the floor app operators would actually use. **The full architecture is in the app.**

---

WATCH FOR  
ONE GOVERNED SOURCE

---

WATCH FOR  
THE FLOOR ACTING ON IT

---

WATCH FOR  
AI YOU CAN AUDIT



# Unplanned Downtime **Reduction**

## **VP Manufacturing Operations**

### THE QUESTION BEHIND THE MANDATE

**Did a plant manager make a different call this shift?**

**Yes.** In the reference build, the model flagged three bearing defects and a screen failure with 18+ days to spare. Real maintenance was scheduled. Real plant managers had options they did not have before.

That is the job: move failures from after-the-fact emergencies to planned work that fits the production calendar.

# The Business Case

Volta's current state and a conservative projection:

|                           |                    |
|---------------------------|--------------------|
| Current downtime per year | <b>1,500 hours</b> |
|---------------------------|--------------------|

---

|               |                 |
|---------------|-----------------|
| Cost per hour | <b>\$22,000</b> |
|---------------|-----------------|

---

|                   |                     |
|-------------------|---------------------|
| Total annual cost | <b>\$33,000,000</b> |
|-------------------|---------------------|

|                                           |                    |
|-------------------------------------------|--------------------|
| If 10% converts to planned<br>(150 hours) | <b>\$3,300,000</b> |
|-------------------------------------------|--------------------|

Achievable with 18 extra days of notice. That is roughly 10 bearing or screen failures caught in the actionable window per year. Your asset mix and failure rates will determine the actual number. We can validate on your historical data in a brief pilot.

# AI Spend at **Scale**

## **Director of Manufacturing IT and Finance**

THE QUESTION BEHIND THE MANDATE

If we roll to eight plants, what does it cost?

**Cost scales with plants and data, not experiments.** The reference environment sits at a 0.5 compute-unit floor and suspends after 300 seconds of idle. Adding a tenth data scientist costs almost nothing.

Eight plants means eight data sources, eight models, eight serving instances. Cost is a function of infrastructure, not friction. We will know the actual spend once we size your data volume.

# What the AI Actually Costs

The cheapest decision we made was choosing not to use a language model for the detection job:

## DETECTION PATH

0

tokens. A gradient-boosted model on 12 numeric features, so sensor volume never buys tokens.

## RATE LIMIT

240

calls per minute at the gateway. A runaway client cannot become a runaway invoice.

## INFERENCE LOGGING

100%

of calls logged to a table in your own catalog, tagged by plant and asset.

Language is used in exactly one place: answering a question on shift. So **token spend tracks how often a person asks, not how much your plant measures**. Two very different cost curves, and choosing the wrong one is how AI programmes get expensive.

# What We Need From You

- ▶ **Access to one plant's data.** Twelve months of telemetry and work orders to validate that the bearing-defect model generalizes to your equipment and to build a plant-specific escalation factor for maintenance decisions.
- ▶ **A technical point person.** Someone who understands your lakehouse setup, infrastructure, and how floor applications connect to your data systems.
- ▶ **Alignment on success.** Early detection in the maintenance backlog? Live production visibility on the floor? Reduced unplanned downtime in the plant manager's metrics? We build for what matters to your business.

# The Next Step

We built this reference implementation on a continuous-process plant. Every number comes from that real build. The architecture transfers directly to discrete manufacturing.

Volta's unplanned downtime problem is solvable.

Let's validate it on your data.